Clipping:

When we have to display a large portion of the picture, then not only scaling & translation is necessary, the visible part of picture is also identified. This process is not easy. Certain parts of the image are inside, while others are partially inside. The lines or elements which are partially visible will be omitted.

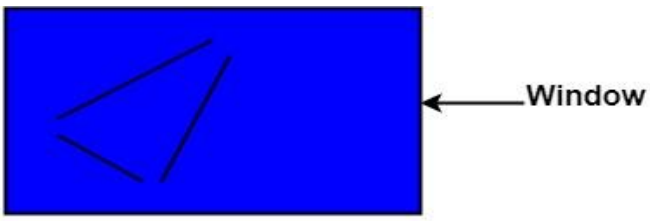
For deciding the visible and invisible portion, a particular process called clipping is used. Clipping determines each element into the visible and invisible portion. Visible portion is selected. An invisible portion is discarded.

Types of Lines:

Lines are of three types:

1. **Visible:** A line or lines entirely inside the window is considered visible
2. **Invisible:** A line entirely outside the window is considered invisible
3. **Clipped:** A line partially inside the window and partially outside is clipped. For clipping point of intersection of a line with the window is determined.

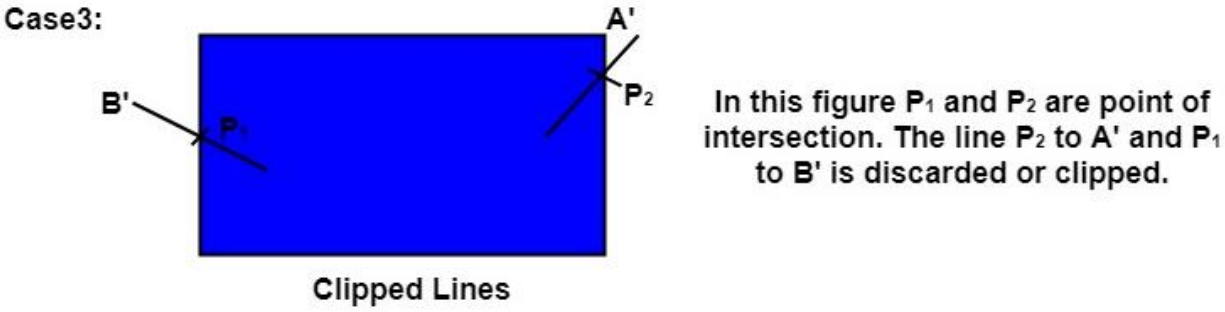
Case1:

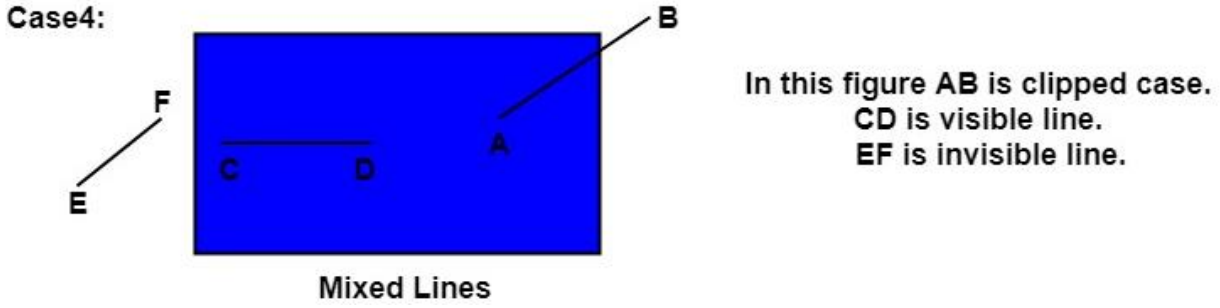


Visible Lines

Case2:

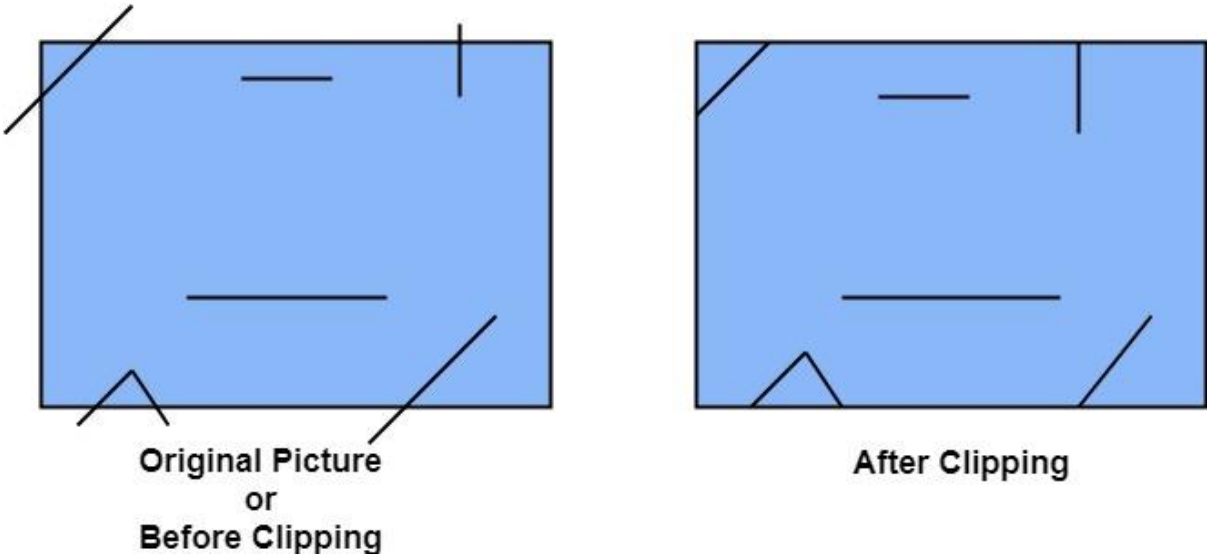
Invisible Lines





Clipping can be applied through hardware as well as software. In some computers, hardware devices automatically do work of clipping. In a system where hardware clipping is not available software clipping applied.

Following figure show before and after clipping



The window against which object is clipped called a clip window. It can be curved or rectangle in shape.

Applications of clipping:

1. It will extract part we desire.
2. For identifying the visible and invisible area in the 3D object.
3. For creating objects using solid modeling.
4. For drawing operations.
5. Operations related to the pointing of an object.
6. For deleting, copying, moving part of an object.

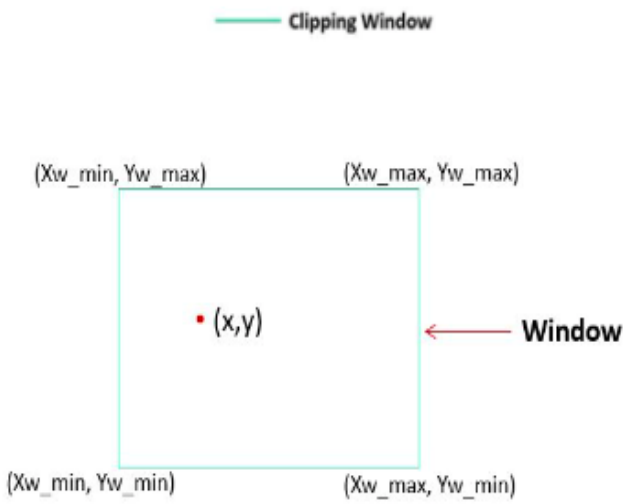
Clipping can be applied to world co-ordinates. The contents inside the window will be mapped to device co-ordinates. Another alternative is a complete world co-ordinates picture is assigned to device co-ordinates, and then clipping of viewport boundaries is done.

Types of Clipping:

1. Point Clipping
2. Line Clipping
3. Area Clipping (Polygon)
4. Curve Clipping
5. Text Clipping
6. Exterior Clipping

Point Clipping:

In point clipping, a point (x,y) is checked whether it falls within the window or not.



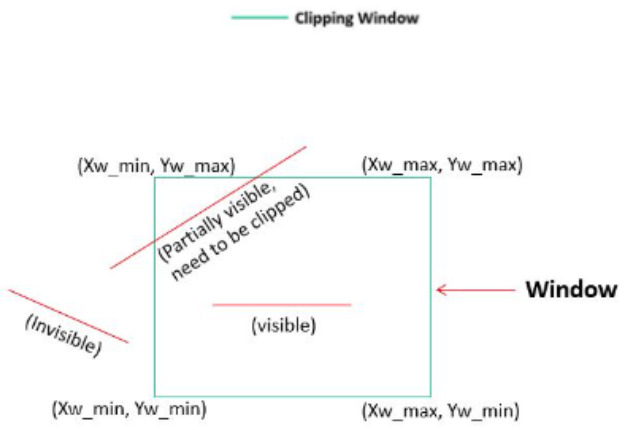
Considering a rectangular window, as mentioned similar to above diagram, suppose we have a point , P = (x,y) if it doesn't meet below condition, then it would not be visible for display.

- A point that lies inside the clipping window is considered as visible and no clipping is required for such case
- If it lies outside the window, then it would be considered as invisible

Further, point clipping is not often used when it's compared to polygon clipping or line clipping.

Line Clipping:

A line clipping comes into play, where the extra portion of a given line that lies outside the window is cut off from the clip region. There is a popular line clipping algorithm named Cohen Sutherland algorithm.



- If a line intersects the clipping window, then the line is required to be clipped as it is considered as partially visible
- If a line falls inside the clipping window, then it is considered as visible
- Otherwise, invisible

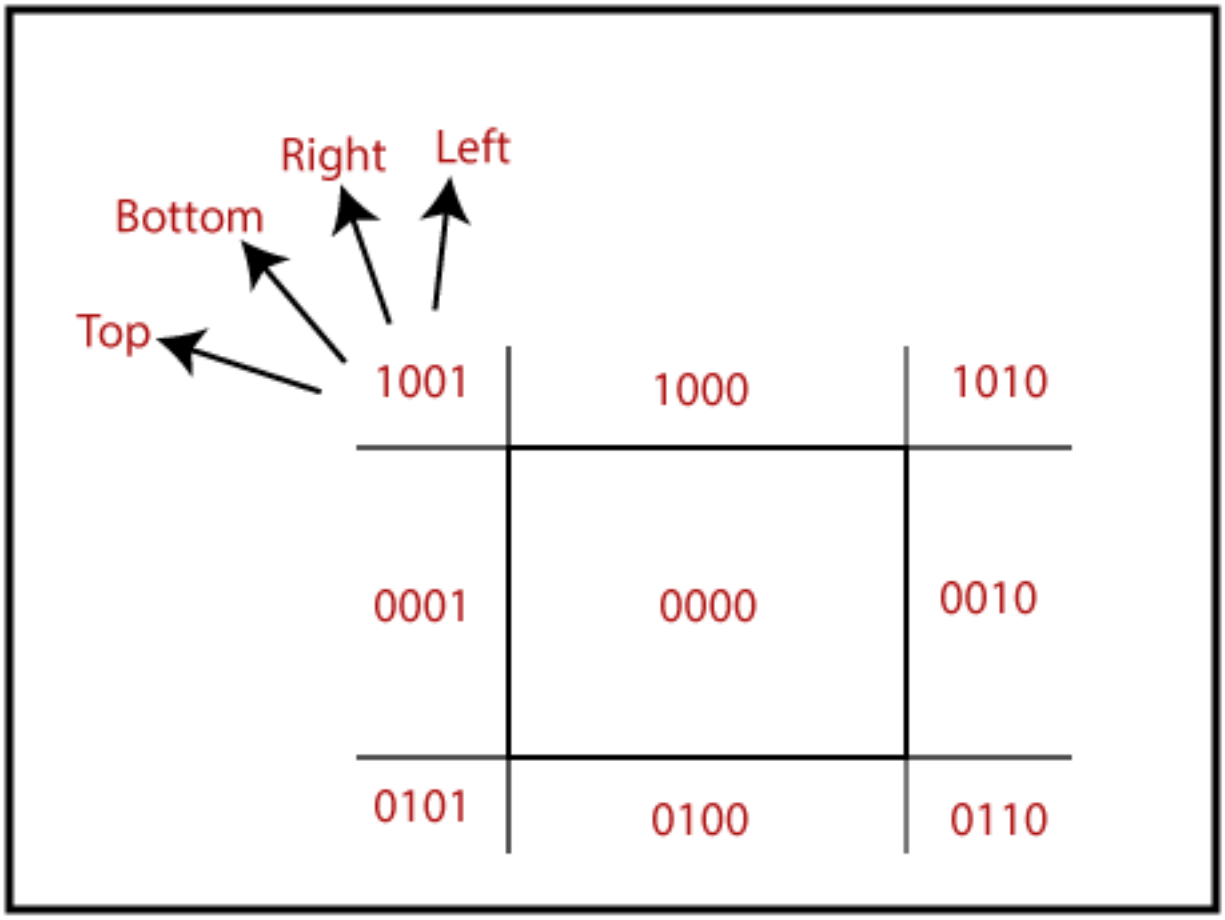
In line clipping, a line segment is first validated whether it is entirely inside or outside clipping window. It is also possible to have a line lies partially inside. In such case, it requires to be clipped out the portion of line segment that falls outside the clipping window.

Cohen Sutherland Line Clipping Algorithm

This algorithm was named by **"Danny Cohen"** and **"Ivan Sutherland."** The algorithm will work in the following steps:

- In this algorithm, we will divide the view pane into nine equal segments (as shown in the below figure) that only serve the viewport.
- Now, we will represent the top, bottom, left, and right corner of the window with 4 bits. This 4 bit can be described with the following point that:

- If an object lies within any particular corner position, that corner value will be 1, else it will be 0.
- The allocation of bits depends on **"TBRL"** (Top, Bottom, Right, Left) rule.
- Suppose, if the point of a line appears in the top-left corner, then according to TBRL, the value is 1001. We will allot the bits as-
- For the top corner, because the object is present at the top corner.
- For the bottom corner, because the object does not lie at the bottom.
- For the right corner, because the object does not lie at the right side.
- For the left corner, because the object lies at the top-left corner.
- In this way, we check TBRL for each segment and allot the bits accordingly.



In Cohen- Sutherland Algorithm we will divide the lines into following Sections-

- **Visible Line:** When both points (starting and ending) of the line are entirely situated inside the window.
- **Invisible Line:** When both points (Starting and ending) of the line are completely situated outside the window.

If we have (x_{min}, x_{max}) and (y_{min}, y_{max}) coordinates of the view pane (window).

Then, it should be described as-

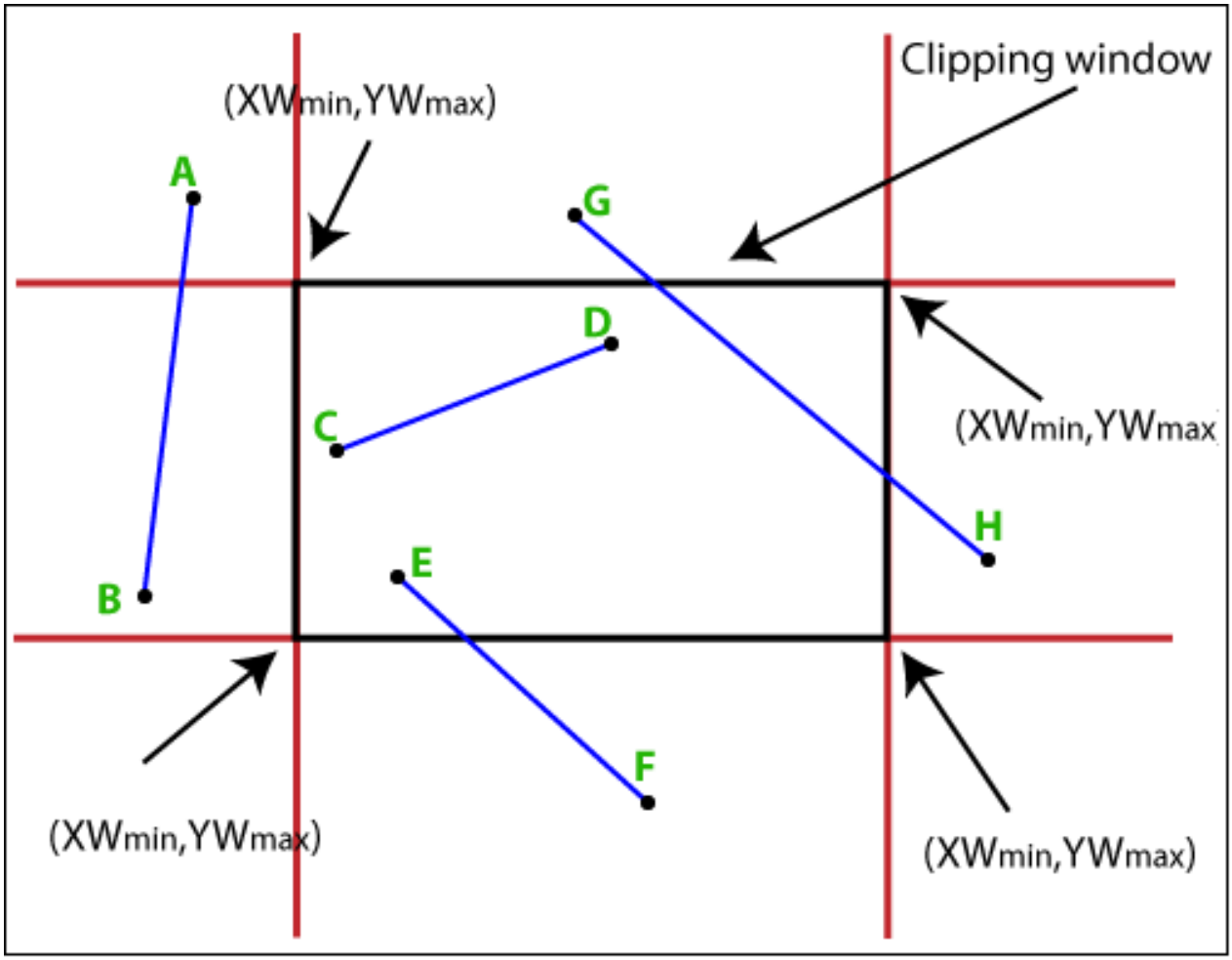
$x_0, x_1 > x_{max}$

$y_0, y_1 > y_{max}$

$x_0, x_1 < x_{min}$

$y_0, y_1 < y_{min}$

- **Clipped Line:** Every line has two endpoints. Let (x_0, y_0) and (x_1, y_1) are points of the line. If one point of the line situated inside the window and the other one is outside the window, then the line is known as Clipped Line.



Algorithm of Cohen-Sutherland Line Clipping:

Step 1: Assign the bit code for both endpoints of the line.

Step 2: Now, implement OR operation on both endpoints of the line.

Step 3: If the OR = 0000,

Then

{The line is acceptable (Visible)}

Else

{Implement AND operation on endpoints}

Then

If AND ≠ 0000

Then

{The line is not acceptable (Invisible)}

Else

AND = 0000

{The line needs to be clip}

Step 4: If a line needs to be clipped, first find an intersection point of all boundaries with the following formula-

$m = (y_1-y_0)\ (x_1-x_0)$

Step 4.1: When the line intersects the left side boundary of the window port.

$y_0 = y_1 + m(x-x_1)$

Here $x = xw_{min}$ (Minimum value of x coordinate)

Step 4.2: When the line intersects the right-side boundary of the window port.

$y_0 = y_1 + m(x-x_1)$

Here $x = xw_{max}$ (Maximum value of x coordinate)

Step 4.3: When the line intersects Top side boundary of the window port.

$x_0 = x_1 + (y-y_1)/m$

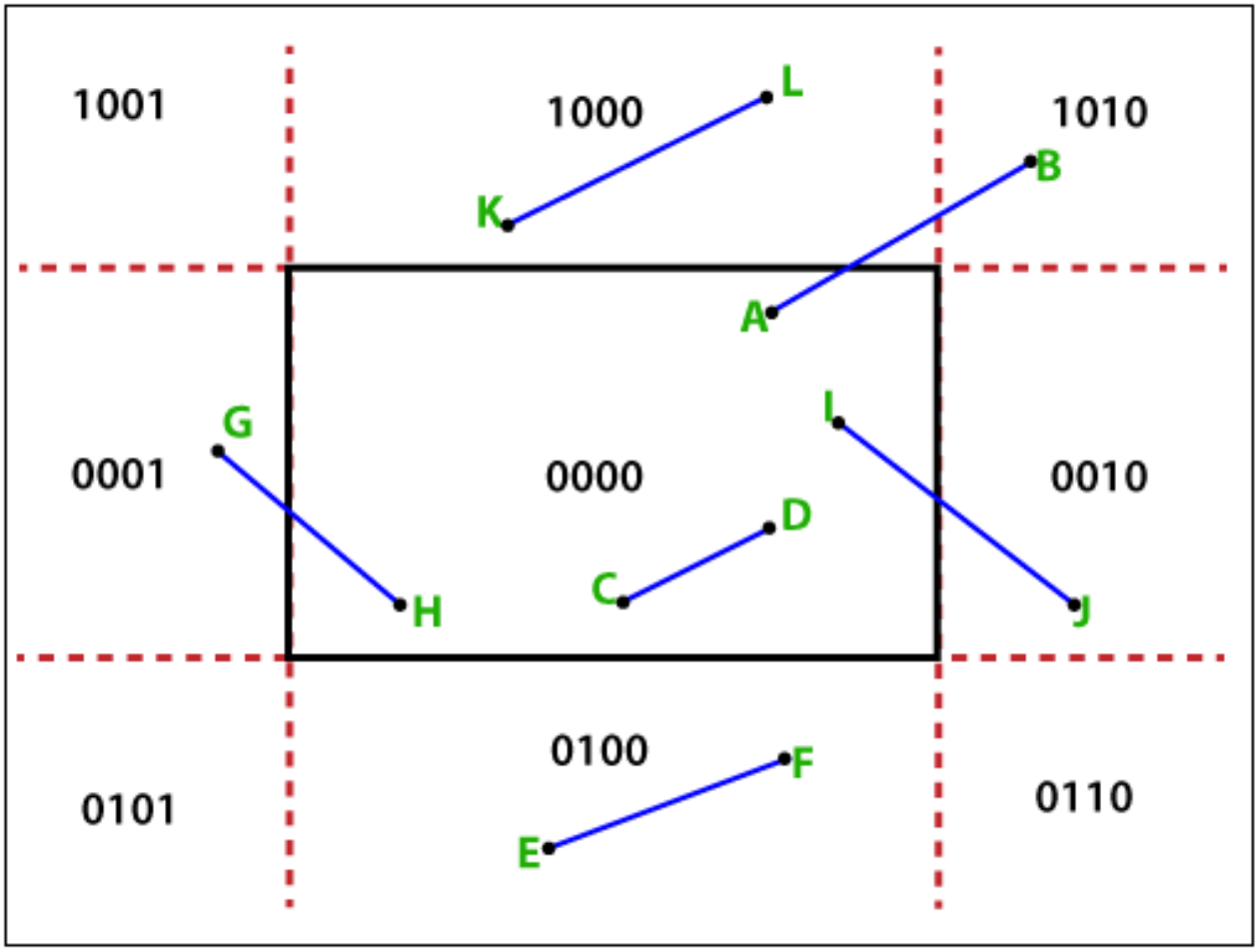
Here $y = yw_{max}$ (Maximum value of y coordinate)

Step 4.4: When the line intersects the bottom side boundary of the window port.

$x_0 = x_1 + (y-y_1)/m$

Here $y = yw_{min}$ (Minimum value of y coordinate)

Example: In the below-mentioned example, we have different lines. The different category of the line-



Line AB is a clipped case.

The line CD is completely visible.

Line EF is completely invisible.

Line GH is a clipped case.

Line KL is completely invisible.

Line IJ is a clipped case.

The endpoints of lines are lies as follows-

A ? 0000

B ? 1010

C ? 0000

D ? 0000

E ? 0100

F ? 0100

G ? 0001

H ? 0000

I ? 0000

J ? 0010

K ? 1000

L ? 1000

Advantages:

1. It is easy to use and implement.
2. We can perform clipping and testing in a particular manner.
3. It is a fast algorithm.

Disadvantages:

1. Sometimes it performs needless clipping.